

A 3D rendering of the text "DL for ENG" in a bold, gold, sans-serif font. The letters are floating on a deep blue ocean with visible ripples and small waves. The sky above is a mix of light blue and grey, with soft, white clouds. The overall scene has a cinematic, high-quality feel.

DL for ENG

LECTURE 1.1 · RECORDED

Python That Runs

You are an engineer who can already program. This is not a course in programming; it is the shortest path from that skill to a Python notebook that runs, plus the one debugging habit that will save you the most time this semester.

Remus Teodorescu

With support from Research Assistant Noman Khan - nomank@energy.aau.dk

Aalborg University · ret@et.aau.dk

Deep Learning for Engineering · 2026

Colab · types and containers · functions · reading a traceback

THE SHAPE OF EVERY CELL YOU WRITE

ONE CELL

```
import numpy as np

def deflection(x, w, L, E, I):
    """Tip deflection of a cantilever, in m."""
    k = 6*L**2 - 4*L*x + x**2
    return w * x**2 * k / (24*E*I)

x = np.linspace(0.0, 2.0, 41)
y = deflection(x, 500.0, 2.0, 210e9, 4.5e-6)
print(y.shape, y[-1])
```

Nine lines, and every idea in this lecture is in them: an import, a function with a docstring, an array, a call, and a print that checks the answer is the right shape before anyone believes the number.

Who This Is For, and How to Watch It

WRITTEN FOR A MATLAB-LITERATE ENGINEER

I assume you can already program. You have written loops, you have indexed a matrix, you have plotted a curve with units on the axes. What you have probably not done is write Python, and the gap is smaller than it looks — perhaps two hours of unfamiliarity spread over twelve weeks of work.

This lecture is recorded, which changes how you should use it. Nothing here is a performance. Stop it whenever you like, and stop it especially at the four points where I say so directly.

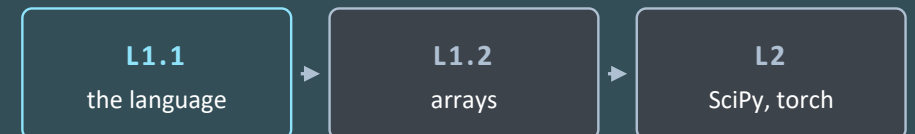
The right way to watch is with a notebook open beside the video. When I show a cell, type it. Typing it wrong and reading the error is worth more than watching me type it correctly — that is the entire argument of the second half of this lecture.

THE ONE THING TO CARRY OUT OF TODAY

An error message is not a failure. It is the most specific, most accurate piece of information you will get all semester, and most people throw it away unread.

WHAT YOU ALREADY KNOW, RENAMED

MATLAB script	a notebook cell, mostly
1-based indexing	0-based, and half-open slices
everything is a matrix	a list is not an array — L1.2
end	indentation, and nothing else
function per file	as many as you like per file
disp / fprintf	print and f-strings
the workspace	the kernel, and it hides from you



Four recorded half-lectures before the live course begins.

Colab, and What It Actually Gives You

A COMPUTER IN A BROWSER TAB, RENTED BY THE HOUR

Google Colab runs a Python notebook on a machine in a data centre and shows it to you in a browser. Everything this course needs — NumPy, matplotlib, pandas, SciPy, PyTorch — is already installed, which removes the commonest reason a student loses the first week.

You need a Google account and nothing else. No install, no administrator rights on a university laptop, and the same environment for everybody, so when something goes wrong it goes wrong the same way for the person you ask.

Two things are worth knowing before they surprise you. The machine is temporary: it is reclaimed after a period of inactivity, and every variable and file on its disk goes with it. And the free tier can refuse you a GPU at busy times — nothing in Part 1 needs one.

SAVE YOUR WORK WHERE IT SURVIVES

File, Save a copy in Drive. The notebook survives; the machine that ran it does not.

WHAT LIVES WHERE

the notebook	in your Drive — permanent
the kernel	a Python process — restartable
variables	in that process — volatile
/content files	on the rented disk — lost
the GPU	if offered, and not needed yet

THE LOCAL ALTERNATIVE

Anaconda or a venv on your own laptop works identically. Slide 22 says how, once, and then this course never mentions it again.

A Cell Is Not a Script

THE ORDER ON THE PAGE IS NOT THE ORDER OF EXECUTION

A script runs from the top to the bottom, once, in a fresh process. A notebook does not. Each cell is submitted to a long-lived Python process — the kernel — in whatever order you press shift-enter, and everything that cell defines stays in that process afterwards.

The number in the brackets to the left of a cell is not its position in the document. It is the count of how many cells the kernel has run, so it tells you the true order and it is the only thing on screen that does.

This is genuinely useful. It is why you can fit a model in one cell and then spend twenty minutes plotting it from another without refitting. It is also the source of a specific and very common failure, which is on the right.

THE FAILURE, IN THREE CELLS

WHAT THE KERNEL BELIEVES

```
In [1]: tau = 60.0
In [2]: T = 22 + 58 * np.exp(-t / tau)

# you edit cell 1 to tau = 45.0
# and do not re-run it

In [3]: print(tau)
        60.0
```

The screen shows forty-five. The kernel holds sixty. Nothing is wrong with your code and nothing raises an error — the number on the screen is simply not the number that produced the figure below it.

This has caused published errata. It is not a beginner's mistake.

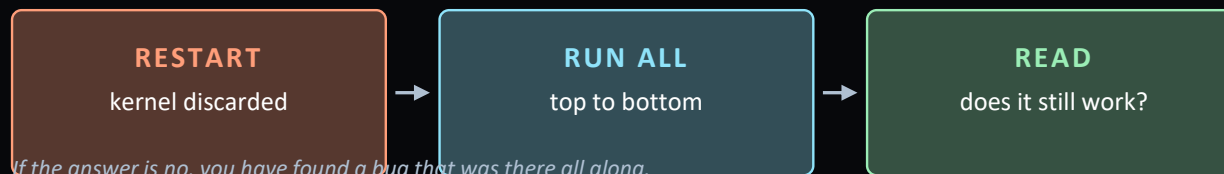
Restart and Run All

ONE MENU ITEM, AND A RULE ABOUT WHEN TO USE IT

Runtime, then Restart and run all. It throws the kernel away, starts a fresh one, and runs every cell from the top in document order. If the notebook works after that, then the document you are looking at really does produce the results underneath it.

Use it before you believe a result, before you hand anything in, and the moment anything surprises you. It costs whatever your notebook takes to run, which in this course is seconds.

The habit it enforces is worth naming: write your notebook so that it survives a restart. That means no cell depends on a variable defined in a cell below it, and no result depends on you having run something twice.



WHEN TO DO IT

before believing	any number you will quote
before submitting	every exercise, every time
when confused	before forming any theory
after editing	a function two cells above
never as a fix	for a bug you have not read

PAUSE THE VIDEO HERE

Open Ex01 notebook 00, run it top to bottom, then restart and run it again. It prints your Python, NumPy and PyTorch versions. If any of them is missing, you have found it in week one rather than in week six.

Values, Types, and Two Divisions

NAMES ARE UNTYPED; VALUES ARE NOT

You do not declare types in Python. A name is bound to a value, and the value knows what it is. Ask it with the `type` function, which you should use far more often than pride suggests.

Four types carry almost everything in this course: `int` for counts and indices, `float` for measurements, `str` for text, and `bool` for the answers to questions. There is no separate double, and a Python float is already what MATLAB calls a double.

The two division operators are the one genuine trap. A single slash, `/`, always gives a float, even when both operands are whole numbers. A double slash, `//`, is floor division and gives an integer. Mixing them up produces an index that is silently one step wrong.

$$\sigma = \frac{F}{A}$$

Four lines of Python, on the right, and the docstring is not optional.

TYPES, AND THE DIVISION TRAP

```
n_samples = 31      # int
tau_s      = 60.0    # float
material    = 'aluminium' # str
converged   = True    # bool
```

```
7 / 2  -> 3.5    (always a float)
7 // 2 -> 3      (floor division)
7 % 2  -> 1      (remainder)
2 ** 10 -> 1024  (power, not ^)
```

THE EQUATION ON THE LEFT

```
def axial_stress(force_N, area_m2):
    """Axial stress in Pa."""
    return force_N / area_m2
```

Printing Things You Can Read

F-STRINGS, AND WHY THE FORMAT MATTERS

An f-string is a string with an `f` in front of it, and anything inside braces is evaluated and inserted. It replaces the whole family of concatenation and formatting calls you might reach for otherwise, and it is what every line of output in this course uses.

Format specifiers go after a colon inside the braces. Three of them cover almost everything: fixed decimal places, scientific notation, and a field width for lining columns up.

This is not cosmetic. A loss printed as a full-precision float is unreadable at a glance, and a table whose columns do not line up hides exactly the pattern you were looking for. You will read thousands of these lines this semester.

PRINT THE SHAPE, ALWAYS

Every time an array is built or reshaped, print its shape on the next line. L1.2 makes the argument in full; start the habit now, before you have anything complicated enough for it to matter.

THE FOUR YOU WILL USE

```
tau, loss = 60.0, 0.00042371

print(f"tau = {tau:.1f} s")
tau = 60.0 s

print(f"loss = {loss:.3e}")
loss = 4.237e-04

print(f"{material:>12s} {tau:8.2f}")
aluminium    60.00
```

Point one `f` for one decimal place, point three `e` for scientific, and a width to line a column up. Everything else you can look up on the day you need it.

Lists, and the Half-Open Slice

TWO CONVENTIONS THAT DIFFER FROM MATLAB

A list is an ordered, mutable sequence written with square brackets. It may hold anything, including a mixture of types, and it can grow with `append`. That flexibility is exactly why it is the wrong container for numerical work, which is L1.2's subject.

Indices start at zero, so the first element is at index zero and the last is at index minus one. Negative indices count backwards, which is genuinely convenient once you stop resenting it.

A slice is written `start:stop`, and the stop is excluded. That is the half-open convention, and it has one very good property: the length of the slice is `stop - start`, with no plus one anywhere, and adjacent slices meet exactly once with no overlap and no gap.

SIX ELEMENTS, AND WHERE INDICES SIT

12.4	13.1	13.9	14.2	14.0	13.6
0	1	2	3	4	5
-6	-5	-4	-3	-2	-1

`y[1:4]` → three elements, and $4 - 1 = 3$

THE FORMS WORTH MEMORISING

<code>y[0]</code>	first	<code>y[-1]</code>	last
<code>y[:3]</code>	first three	<code>y[3:]</code>	the rest
<code>y[::2]</code>	every other	<code>y[::-1]</code>	reversed
<code>len(y)</code>	6	<code>y.append(v)</code>	

A Name Is Not a Box

ASSIGNMENT BINDS A NAME; IT DOES NOT COPY

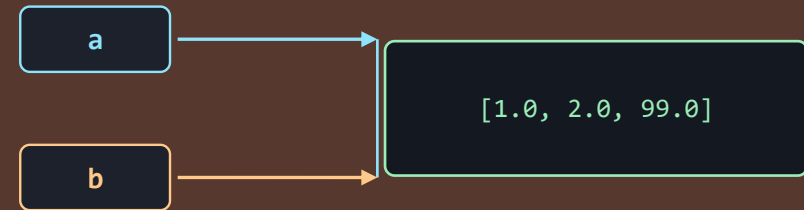
Writing `b = a` does not make a second list. It makes a second name pointing at the same list. Change the list through either name and both names show the change, because there was only ever one list.

This is not a quirk. It is how every mutable object in Python behaves, including NumPy arrays and PyTorch tensors, and it is a deliberate design choice: copying a large array on every assignment would make the language unusably slow.

The consequence for you is a rule and a habit. The rule: if you want an independent copy, ask for one, with the `copy` method or with `list` of the original. The habit: when a value changes and you cannot see what changed it, suspect that two names are pointing at one object before you suspect anything else.

Immutable values — numbers, strings, tuples — cannot show this behaviour, because there is no operation that changes them in place. That is most of the argument for using a tuple when you do not intend to modify something.

TWO NAMES, ONE LIST



One object. Two labels. Nothing was duplicated at any point.

SAY WHAT YOU MEAN

```
b = a           # same list
c = a.copy()    # a new list
d = list(a)     # also a new list
e = a[:]        # also, and less clear
```

Dictionaries: Names, Not Positions

THE RIGHT CONTAINER FOR A SET OF NAMED QUANTITIES

A dictionary maps keys to values and is written with braces. Look a value up by its key rather than by its position, and add a new key simply by assigning to it.

This is the container for material properties, for the settings of a run, for anything where a number has a name. The alternative — a list where you have to remember that index two is Young's modulus — is a bug waiting for the day you insert something.

Two habits. Iterate with the `items` method, which gives you the key and the value together. And use the `get` method with a default when a key might legitimately be absent, because indexing a missing key raises a `KeyError`, which is the correct behaviour and not always the behaviour you want.

You will meet dictionaries constantly from Ex03 onwards, because that is how a training configuration and a set of results are both carried around.

A MATERIAL, AS A DICT

```
alu = {
    "name": "aluminium",
    "E_Pa": 69.0e9,
    "rho": 2700.0,
    "c_p": 900.0,
}

alu["E_Pa"]           69000000000.0
alu.get("k", 205.0)   205.0
"rho" in alu          True
list(alu)              the keys

for key, value in alu.items():
    print(f"{key:>6s} {value}")
```

Indexing a key that is not there raises `KeyError`. That is a feature.

Tuples, Unpacking, and Choosing

AN IMMUTABLE LIST, AND WHY THAT IS USEFUL

A tuple is written with round brackets and cannot be changed after it is made. Use it for a fixed group of things: a coordinate, a pair of return values, a shape.

That last one matters immediately. The shape of a NumPy array is a tuple, and so is the shape of a PyTorch tensor, so you will read tuples every day from the next lecture onwards.

Unpacking is the reason tuples feel pleasant rather than restrictive. Write two names on the left of an assignment and Python takes the tuple apart for you. A function that returns two things returns a tuple, and you unpack it at the call site.

LIST

ordered, changes size

TUPLE

fixed group, never changes

DICT

looked up by name

Three containers. Choose by how you will get things out again, not by taste.

UNPACKING, WHICH YOU USE HOURLY

```
point = (0.0, 2.0)
x, y = point

def fit_line(x, y):
    """Return slope and intercept."""
    ...
    return slope, intercept

a, b = fit_line(x, y)

n_rows, n_cols = data.shape
t, T = load_cooling_data()
```

The last two lines are from Ex03, and they are the whole reason this slide exists.

Indentation Is the Syntax

NO END, NO BRACES, AND NO ARGUING ABOUT IT

A block in Python is defined by how far it is indented. There is no end keyword and there are no braces. The line introducing a block ends in a colon, and everything indented under it belongs to the block.

The universal convention is four spaces, and every editor you will use in this course inserts them for you. Mixing tabs and spaces produces an `IndentationError` or, worse, a block that is not the block you meant.

The for loop iterates over the things themselves, not over a range of indices. This reads better and it is also less error-prone, because most index arithmetic exists only to undo the decision to loop over indices in the first place. When you genuinely need the position as well, `enumerate` gives you both.

The `elif` keyword is else-if compressed, and it exists because a chain of nested else blocks marches off the right of the screen.

THE THREE SHAPES

```
if strain > 0.002:
    verdict = "yielded"
elif strain > 0.001:
    verdict = "approaching yield"
else:
    verdict = "elastic"
```

```
for reading in gauges:
    total += reading

for i, reading in enumerate(gauges):
    print(f"gauge {i}: {reading:.2f}")
```

```
while residual > tol and step < 500:
    residual = iterate(residual)
    step += 1
```

Always give a while loop a second exit condition.

Comprehensions, and Their Limit

A LOOP THAT BUILDS A LIST, WRITTEN ON ONE LINE

A list comprehension is a for loop that produces a list, written inside the brackets it produces. It is idiomatic Python and you will read it in every library you touch, so you need to recognise it even if you choose not to write it.

It takes an optional filter — an if at the end — which is how you say the samples above the threshold in one clause instead of three lines.

Now the honest limit, which the next lecture measures. A comprehension is not faster than the loop it replaces in any way that matters. The arithmetic still happens one Python object at a time. It is more readable, not fundamentally quicker, and if speed is your problem the answer is a NumPy expression rather than a tidier loop.

So write comprehensions when they are clearer, which is when they fit on one line and do one thing. A comprehension with two filters and a nested loop inside it is a loop wearing a disguise.

THE SAME THING, TWICE

THE LOOP

```
squares = []  
for v in readings:  
    squares.append(v ** 2)
```

THE COMPREHENSION

```
squares = [v ** 2 for v in readings]  
  
large = [v for v in readings if v > 3.0]  
  
names = {k: v for k, v in alu.items()}
```

AND THE MEASUREMENT, NEXT LECTURE

Loop and comprehension differ by a few per cent. The NumPy version of the same arithmetic is faster by a factor in the tens to low hundreds.

Functions, and What Goes in the Docstring

THE UNIT OF REUSE, AND THE UNIT OF TESTING

Define a function with `def`, a name, the arguments in brackets, and a colon. The body is indented. A return statement hands a value back; a function with no return hands back the value `None`, which is a real value and will surprise you if you try to do arithmetic with it.

Arguments can have defaults, and arguments can be passed by name at the call site. Passing by name is the right choice whenever a call has more than about three arguments, because a bare list of five numbers in brackets is unreadable six weeks later.

The docstring is the string on the first line of the body. Put the units in it. In an engineering course this is not a style preference: a function that returns a deflection is useless unless the reader knows whether the answer is in metres or millimetres, and the only place that fact can live is the docstring.

UNITS IN THE DOCSTRING, UNITS IN THE NAME

Ex_1_core writes `force_N` and `area_m2` rather than `F` and `A`. It is slightly ugly and it has never once caused a unit error.

ONE FUNCTION, WRITTEN PROPERLY

```
def beam_deflection(x_m, load_N_per_m=500.0,
                   length_m=2.0):
    """Deflection of a cantilever, in metres.

    x_m is the distance from the fixed end.
    Uniform load, small-deflection theory.
    """
    L = length_m
    return (load_N_per_m * x_m**2 *
            (6*L**2 - 4*L*x_m + x_m**2)) / (24*E*I)

y = beam_deflection(1.0, load_N_per_m=250.0)
```

The keyword at the call site is what makes the last line readable.

One Trap Worth a Whole Slide

A DEFAULT ARGUMENT IS EVALUATED ONCE, NOT PER CALL

When Python reads a def statement it evaluates the default values there and then, once, and stores them on the function. It does not re-evaluate them on each call.

For a number or a string that makes no difference, because those cannot be changed. For a list or a dictionary it makes every difference: all calls share one object, and anything appended to it in one call is still there on the next.

The fix is a convention rather than a language feature. Default to None, and build the real default inside the body. Two extra lines, and it removes the entire class of problem.

I am giving this a whole slide because it is the kind of bug that produces a plausible wrong answer rather than an error. Your history list quietly accumulates results from previous runs, your mean is computed over more samples than you supplied, and nothing anywhere raises.

THE BUG

```
def log(value, history=[]):  
    history.append(value)  
    return history  
  
log(1.0)    -> [1.0]  
log(2.0)    -> [1.0, 2.0]
```

THE FIX

```
def log(value, history=None):  
    if history is None:  
        history = []  
    history.append(value)  
    return history
```

Imports, Namespaces, and Where Python Looks

THE PREFIX IS INFORMATION, NOT NOISE

An import binds a module to a name. Writing `np dot linspace` tells the reader that `linspace` comes from NumPy, and that is worth the four characters — particularly once `torch` is in the room and half the names exist twice.

The four aliases in this course are fixed and universal: `np`, `plt`, `pd` and `torch`, which is never abbreviated. Use them, because every example you find online uses them.

The `from` form imports a name directly into your namespace. It is right for a handful of specific things and wrong as a habit, and the star form, which imports everything, is wrong always: it makes it impossible to tell where a name came from, and a later import silently replaces an earlier one.

Where does Python look? First the directory the notebook is running in, then the installed packages. That order is why a file of your own named `numpy dot py` breaks every import in the notebook, with a message that does not mention your file.

THE FIXED ALIASES

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import torch

from scipy.integrate import solve_ivp

from numpy import *    # never
```

THE SEARCH ORDER, AND ITS TRAP

Your own directory first, installed packages second. A file called `numpy.py` in your folder shadows the real NumPy, and the error you get will talk about a missing attribute rather than about your file.

What a Traceback Actually Is

THE MOST SPECIFIC INFORMATION YOU GET ALL SEMESTER

When something goes wrong, Python prints a traceback: the chain of function calls that led to the failure, oldest first, with the error itself at the very bottom.

Most people see a wall of red text, feel a small drop in confidence, and go back to staring at their own code. That is exactly backwards. The traceback already contains the answer. It has simply been printed in an order nobody explained.

On the right is a real one, produced by calling a function in the Ex01 module that raises deliberately. Twelve lines. Look at it for a moment before I say anything about it, and notice how much of it is your code and how little is anybody else's.

WHY THIS GETS THREE SLIDES

It is the single highest-value debugging skill in the course. A student who reads tracebacks fluently solves in ninety seconds what otherwise costs an evening and a message to a classmate.

A REAL TRACEBACK, UNEDITED

```
Traceback (most recent call last):
  File "<ipython-input-7>", line 1, in <module>
    traceback_demo()
  File "Ex_1_core.py", line 511, in traceback_demo
    outer()
  File "Ex_1_core.py", line 502, in outer
    return middle([2.0, 4.0, 6.0])
  File "Ex_1_core.py", line 505, in middle
    return inner(values, 5)
  File "Ex_1_core.py", line 508, in inner
    return values[index]
IndexError: list index out of range
```

Twelve lines. Five of them are frames — a file, a line number, and the source line that was running. One of them is the error. The next two slides take this apart in the order that makes it useful.

Read It from the Bottom Up

THREE QUESTIONS, IN THIS ORDER

First: what went wrong? That is the last line, and only the last line. An exception type and a message, in English. Read it literally — list index out of range means an index was outside a list, and nothing more subtle than that.

Second: where? That is the frame directly above the last line. A file, a line number, and the offending source line itself. In this case, values indexed by index, inside a function called inner.

Third: how did we get there? That is everything above, read upwards. Scan up until you find the last frame that is in code you wrote. That frame is nearly always the line you have to change, even when the exception was raised deep inside a library.

Here the answer is the call in middle, which asks for element five of a list with three elements. The failure is in inner; the mistake is in middle. Those are different lines, and confusing them is why people edit the wrong function.

THE SAME TWELVE LINES, ANNOTATED

```
Traceback (most recent call last):
  File "<ipython-input-7>", line 1, in <module>
    traceback_demo()
  File "Ex_1_core.py", line 511, in traceback_demo
    outer()
  File "Ex_1_core.py", line 502, in outer
    return middle([2.0, 4.0, 6.0])
  File "Ex_1_core.py", line 505, in middle
    return inner(values, 5)
  File "Ex_1_core.py", line 508, in inner
    return values[index]
IndexError: list index out of range
```

3 · read upwards the mistake — index 5

2 · the frame above where it broke — inner

1 · the last line what broke — IndexError

The Six You Will Actually Meet

EACH ONE MEANS ONE THING, AND THE MESSAGE SAYS WHICH

NameError

A name that does not exist yet. Usually a typo, or a cell you have not run — the kernel problem again.

IndexError

An index outside a sequence. `KeyError` is its dictionary twin, and means the key is absent.

ValueError

The type is right and the value is not. In NumPy this almost always means shapes that will not broadcast.

TypeError

An operation between things that do not combine. A list plus a number; a `None` where a float was expected.

AttributeError

The object has no such method. Very often the object is `None`, because a function returned nothing.

ZeroDivisionError

Exactly what it says. In array code you get `inf` or `nan` instead, silently, which is worse.

AND THE ONE THAT DOES NOT RAISE AT ALL

`Ex01`'s `broken_average` sums from index one instead of index zero and divides by the full length. It returns a number. The number is wrong. No traceback exists for this, and the only defence is checking one case you can compute in your head.

The Error That Prints a Number

WORSE THAN A TRACEBACK: IT LOOKS LIKE SUCCESS

The function on the right computes the mean of a list. It contains one bug, and it is a bug you will write yourself at least once this semester: the loop starts at index one, so the first sample is never added, but the division is still by the full count.

Give it the numbers two, four and six and it returns three point three three, where the answer is four. Nothing raises. There is no red text and no line number, and if you had not known the answer you would have written it down.

The defence is a habit, not a tool. Every function you write gets tested once on a case whose answer you already know — three numbers, a single element, a zero. It takes fifteen seconds and it is the only thing that catches this class of error.

That is what the check cells in every exercise notebook are doing. They are not marking you. They are the habit, written down, so that you acquire it before you are working on something where nobody knows the right answer.

ONE BUG, NO EXCEPTION

```
def broken_average(values):  
    total = 0.0  
    for i in range(1, len(values)):  
        total += values[i]  
    return total / len(values)  
  
broken_average([2., 4., 6.]) -> 3.333
```

$$\bar{y} = \frac{1}{N} \sum_{i=0}^{N-1} y_i$$

THE FIFTEEN-SECOND TEST

Mean of a single element is that element. Mean of three equal numbers is that number. Either one catches this.

What to Do, in Order

A PROCEDURE, SO YOU NEED NOT INVENT ONE

Read the last line of the traceback and say out loud what it means. Then find the last frame that is your code. Then print the things on that line — the value, the type, and from next lecture the shape — before forming any theory at all.

The order matters. The commonest failure mode among competent engineers is to form a theory first and then look for evidence supporting it, which in a language you are new to is almost always the wrong theory.

Then change one thing. Not three. If you change three things and the problem goes away you have learned nothing, and you will meet it again next week.

And when you are genuinely stuck, reduce the case. Four elements instead of two hundred thousand, one time step instead of five hundred. Almost every bug survives the reduction, and in the reduced case you can check the answer by hand.

IN ORDER, AND WITHOUT SKIPPING

- 1 read the last line, literally
- 2 find the last frame you wrote
- 3 print the value, the type, the shape
- 4 say what you expected, and why
- 5 change exactly one thing
- 6 shrink the case until you can check it
- 7 restart and run all, then believe it

PAUSE THE VIDEO HERE

Ex01 notebook 01, section six, asks you to diagnose a real traceback before reading the explanation. Do it in that order.

Virtual Environments in One Slide

WHAT THEY ARE FOR, AND WHY THIS COURSE DOES NOT

Python installs packages globally by default, so two projects on one laptop share one set of versions. Sooner or later one project needs an old version of something and the other needs a new one, and installing either breaks the other.

A virtual environment is a private directory of packages belonging to one project. Create it, activate it, install into it, and the rest of the machine is untouched. Three commands, on the right.

You do not need this for this course, because Colab hands you a fresh environment every session and everything is already installed. I am telling you because you will need it the first time you write Python for a real project, and because a colleague will assume you know what the word means.

If you do work locally, make an environment for this course, install the four packages, and never think about it again. This is the only slide in twelve lectures that mentions the subject.

THE THREE COMMANDS

```
python3 -m venv .venv
source .venv/bin/activate
pip install numpy scipy pandas \
        matplotlib torch

# .venv\Scripts\activate on Windows
```

WHAT THIS COURSE ASSUMES

Google Colab, with no installation of any kind. Everything in Part 1 runs on the free tier without a GPU, and the largest network you will train in Part 1 has roughly two thousand parameters.

Ex01 — Python Fundamentals

FOUR NOTEBOOKS, AND THE FIRST TWO ARE THIS LECTURE

Notebook 00 is a read-only environment check. Run it first and run it today, because a missing PyTorch found in week one costs nothing and the same discovery in week six costs an exercise session.

Notebook 01 is today's lecture as work you do rather than watch: types, containers, control flow, functions, imports, and a section on reading a traceback that is deliberately the longest in the notebook.

Notebooks 02 and 03 belong to the next half-lecture, so leave them until you have watched it.

Every TODO cell in Ex01 is preceded by several paragraphs explaining what is being asked and what a correct answer looks like. That is deliberate, and it is because these two lectures are recorded — you have no room to ask a question in the middle, so the notebook has to answer it in advance. The prose thins out from Ex03 and is gone by Ex07.

THE NOTEBOOKS

00	environment check — read only
01	today: types, functions, tracebacks
02	L1.2: arrays and broadcasting
03	L1.2: engineering figures

The running example is a two-metre steel cantilever under a distributed load, with a dial gauge good to about four hundredths of a millimetre. It appears in all four notebooks, so the shapes always mean something physical.

PAUSE THE VIDEO HERE, AND FOR LONGER

Ex01 notebooks 00 and 01, end to end. About an hour if you type rather than read. Then come back for L1.2, which needs the vocabulary.

Key Takeaways

1

A cell is not a script

The kernel remembers everything, in the order you ran it. Restart and run all before you believe anything.

2

Zero-based, half-open

The first element is at index zero and a slice excludes its stop, so its length is stop minus start.

3

A name is not a box

Assignment binds a second name to the same object. Ask for a copy when you want one.

4

Read the traceback bottom up

What, then where, then how you got there. The last frame you wrote is nearly always the line to change.

5

The worst errors do not raise

A plausible wrong number has no traceback. Test every function once on a case you can do in your head.

Next — L1.2: arrays, shapes, and the one rule that explains most of the bugs in the second half of this course.

References and Further Reading

THIS LECTURE HAS NO TEXTBOOK, AND THAT IS DELIBERATE

The two books for this course — Prince's Understanding Deep Learning and Goodfellow, Bengio and Courville's Deep Learning — assume you can already write Python and neither of them teaches it. That is why L1 and L2 exist as recorded lectures rather than as reading.

The official Python tutorial is the best free introduction there is, and sections three, four and five cover everything on today's slides in about ninety minutes of reading. The NumPy and matplotlib documentation take over from the next lecture.

Do not go looking for a book. You do not need one, and the time is better spent **in the notebooks.**

IF YOU WANT ONE THING TO READ

The official tutorial, sections 3 to 5, at docs.python.org. Free, short, accurate and maintained by the people who wrote the language.

Everything else you need this semester is in the exercise notebooks, which are written to be read rather than merely executed.

IN THIS ORDER

today	Ex01 notebooks 00 and 01
if stuck on syntax	docs.python.org tutorial §3–5
before L1.2	nothing — just watch it
before L2.2	Ex01 notebook 02, done properly
never	a 900-page Python book



The four recorded half-lectures, and where you are in them.

NEXT

L1.2 — NumPy, and the Shape of Everything

Today the language. Next time the object every later lecture is actually made of — the array — and one rule about shapes that explains most of the bugs you will hit in weeks seven to twelve.

BEFORE THE NEXT RECORDING

run	Ex01 notebook 00, all the way
work	Ex01 notebook 01 — about an hour
practise	reading one traceback aloud
do not	start notebook 02 yet

Notebook 02 is the broadcasting notebook and it is the one to protect if you run short of time. Watch L1.2 first; it exists to make that notebook take twenty minutes instead of two hours.